

实验报告

实验题目：简单函数绘图语言语法分析器的设计与实现

一、概述

本实验在实验一词法分析器基础上，完成了简单函数绘图语言的语法分析器设计与实现。语法分析器采用递归下降法，支持函数定义语句、绘图语句及带优先级的表达式分析，并可输出树形语法结构。

主要完成内容如下：

1. 设计并实现无左递归的上下文无关文法。
2. 实现 Parser 类及核心递归函数（`parse_program`、`parse_stmt`、`parse_expr` 等）。
3. 实现词法 - 语法联动，直接读取词法分析器 Token 流。
4. 实现多类语法错误检测，支持定位并输出明确错误信息。
5. 通过 5 组测试样例验证正确性与健壮性。

二、实验方案

1. 语法结构设计

根据实验任务，将目标语言分为以下语法模块：

- Program: 程序入口。
- StmtList: 语句序列。
- Stmt: 语句分派（FuncDef / PlotStmt）。
- Expr: 表达式层次（Expr > Term > Factor > Atom）。

文法如下：

- Program -> StmtList EOF
- StmtList -> Stmt ; StmtList | epsilon
- Stmt -> FuncDef | PlotStmt
- FuncDef -> KEYWORD(func) IDENTIFIER LPAREN IDENTIFIER RPAREN OPERATOR(=) Expr
- PlotStmt -> KEYWORD(plot) IDENTIFIER LPAREN IDENTIFIER RPAREN KEYWORD(range) LBRACKET Number COMMA Number RBRACKET
- Expr -> Term Expr'
- Expr' -> + Term Expr' | - Term Expr' | epsilon
- Term -> Factor Term'
- Term' -> * Factor Term' | / Factor Term' | epsilon

- Factor $\rightarrow$ Atom $\wedge$ Factor | Atom
- Atom $\rightarrow$ IDENTIFIER | Number | LPAREN Expr RPAREN
- Number $\rightarrow$ INTEGER | FLOAT

2. 递归下降实现思路

1. 每个非终结符定义一个同名解析函数。
2. 使用 `current_token` 作为单符号前瞻。
3. 使用 `eat()` 执行严格匹配。
4. 对表达式采用分层函数，保证优先级和结合性：
 - `parse_expr` 负责加减。
 - `parse_term` 负责乘除。
 - `parse_factor` 负责幂运算（右结合）。

3. 程序流程

输入源代码 $\rightarrow$ Lexer 产生 Token $\rightarrow$ Parser 递归匹配 $\rightarrow$ 生成 AST $\rightarrow$ 打印语法树或输出语法错误。

三、实验过程及结果分析

1. 核心代码（节选）

```
class Parser:
    def __init__(self, lexer):
        self.lexer = lexer
        self.current_token = self.lexer.get_next_token()

    def advance(self):
        self.current_token = self.lexer.get_next_token()

    def eat(self, token_type, token_value=None):
        tok = self.current_token
        if tok.type != token_type:
            raise SyntaxError(f"期望 {token_type}, 实际 {tok.type}, 位置 {tok.pos}")
        if token_value is not None and tok.value != token_value:
            raise SyntaxError(f"期望 {token_value}, 实际 {tok.value}, 位置 {tok.pos}")
        self.advance()

    def parse_program(self):
```

```

stmts = []

while self.current_token.type != 'EOF':
    stmts.append(self.parse_stmt())
    self.eat('SEMICOLON')

return ('Program', stmts)

def parse_stmt(self):
    tok = self.current_token
    if tok.type == 'KEYWORD' and tok.value == 'func':
        return self.parse_func_def()
    if tok.type == 'KEYWORD' and tok.value == 'plot':
        return self.parse_plot_stmt()
    raise SyntaxError(f"非法语句起始符 {tok.value}, 位置 {tok.pos}")

def parse_factor(self):
    left = self.parse_atom()
    if self.current_token.type == 'OPERATOR' and self.current_token.value == '^':
        self.eat('OPERATOR', '^')
        right = self.parse_factor() # 右结合
        return ('Pow', left, right)
    return left

```

2. 测试过程与结果

测试用例 1（合法）

输入：

```
func f(x) = x^2 + 3.14;
```

结果：

- 语法分析成功。
- 生成 Program -> FuncDef -> Expr 结构。

测试用例 2（合法）

输入：

```
func g(x) = (x + 1) * (x - 1);
```

```
plot g(x) range [0, 10];
```

结果：

- 语法分析成功。

- 能正确识别多语句程序与括号表达式。

测试用例 3（合法）

输入：

```
func h(x) = x^2^3 - 2*x/5;
```

```
plot h(x) range [0, 1.5];
```

结果：

- 语法分析成功。
- 幂运算按右结合解析为 $x^{(2^3)}$ 。

测试用例 4（非法：缺少分号）

输入：

```
func f(x) = x + 1
```

```
plot f(x) range [0, 5];
```

结果：

SyntaxError: 期望 SEMICOLON, 实际 KEYWORD, 位置 16

测试用例 5（非法：括号不匹配）

输入：

```
plot f(x range [0, 10];
```

结果：

SyntaxError: 期望 RPAREN, 实际 KEYWORD(range), 位置 8

测试用例 6（非法：关键字误用）

输入：

```
plto f(x) range [0, 10];
```

结果：

SyntaxError: 非法语句起始符 plto, 位置 0

3. 调试分析

1. 问题一：Expr 直接左递归导致递归函数死循环
 - 原因：早期使用 $\text{Expr} \rightarrow \text{Expr} + \text{Term}$ 形式。
 - 解决：改写为 $\text{Expr} \rightarrow \text{Term Expr}'$ 。
2. 问题二：幂运算结合性错误
 - 现象： x^{2^3} 被错误解析为 $(x^2)^3$ 。

- 解决: `parse_factor()` 中递归解析右侧因子, 改为右结合。
- 3. 问题三: 错误信息不直观
 - 现象: 仅提示“语法错误”, 无法定位。
 - 解决: 在 `Token` 中加入 `pos`, 报错信息中输出“期望/实际/位置”。

四、总结与体会

1. 语法分析器的核心是“文法设计优先于编码实现”。文法层次清晰后, 递归下降代码实现会非常自然。
2. 处理表达式时, 优先级与结合性是正确性的关键, 不可仅依赖顺序匹配。
3. 错误提示质量直接影响调试效率, 输出精确位置和期望符号可以显著缩短排错时间。
4. 与实验一联动后, 完整体会了“字符流 -> `Token` 流 -> 语法结构”的编译前端基本流程, 为后续语义分析和中间代码生成打下了基础。